Спам-фильтр: хеш-таблица и k ближайших соседей

Цель работы: применить собственную хеш-таблицу из ДЗ1 для построения словаря «слово → индекс» в модели «мешок слов» и исследовать, как на неё влияет хеш-функция. Реализовать три способа выбора k ближайших объектов — полной сортировкой, кучей размера k и быстрым выбором — и экспериментально сопоставить их скорость роста с теоретическими оценками. Применить полученные структуры в классификаторе писем методом k ближайших соседей.

Теоретические сведения

Классификация методом k ближайших соседей

Классификация — задача обучения с учителем: есть объекты с известными метками классов (обучающая выборка), и для нового объекта нужно предсказать метку. В фильтрации почты классы — нормальное письмо (ham), рекламная рассылка (spam), фишинг (phish).

Метод k ближайших соседей (k-NN) не строит модели — он хранит обучающую выборку целиком. Для нового объекта вычисляется расстояние до каждого обучающего, выбираются k ближайших, и объекту присваивается метка, за которую проголосовало большинство соседей. Классификация одного запроса поэтому требует просмотра всей выборки, и именно выбор k ближайших из n кандидатов — алгоритмическое ядро работы.

Гиперпараметр k регулирует чувствительность: при малом k решение определяется одним-двумя соседями и чувствительно к шуму, при большом — сглаживается, но мелкие классы «тонут» в крупных.

Мешок слов и словарь

Чтобы считать расстояния, текст нужно превратить в числовой объект. Модель «мешок слов» (bag of words) отбрасывает порядок слов: письмо представляется **множеством** входящих в него слов. Для этого текст разбивается на слова, слова приводятся к нижнему регистру, знаки препинания отбрасываются.

Хранить слова строками и сравнивать их строками дорого. Поэтому строится **словарь** — отображение «слово → индекс»: каждое новое слово получает следующий свободный номер. После этого письмо — это отсортированный массив целых индексов, а пересечение двух писем считается одним проходом слиянием двух отсортированных массивов за $O(|A| + |B|)$.

Словарь — классическое применение хеш-таблицы: на каждое слово каждого письма выполняется одна операция «найти или добавить», и от того, работает ли она за $O(1)$, зависит время построения всего мешка слов. Сама таблица у вас уже есть — это хеш-таблица из ДЗ1; здесь она получает другую хеш-функцию и реальную нагрузку.

Расстояние между письмами

В базовом уровне расстояние между письмами A и B — число слов, входящих ровно в одно из двух множеств (мощность симметрической разности). Это то же самое, что манхэттенское расстояние между 0/1-векторами мешка слов:

$$d(A, B) = |A| + |B| - 2 \cdot |A \cap B|$$

Расстояние целое, поэтому результат работы программы определён однозначно и проверяется автоматически без допусков на погрешность вычислений. Платой за простоту становится известный недостаток: короткое письмо оказывается «близким ко всем», поскольку у него мало слов, которые могли бы не совпасть. На письмах из публичных тестов это хорошо видно; исправление — нормированные метрики (Жаккар, косинусная) — вынесено на повышенный уровень.

Хеш-таблицы

Хеш-таблица хранит пары «ключ — значение» в массиве из m ячеек (корзин). Хеш-функция h переводит ключ в целое число, номер корзины — h(ключ) mod m. Если хеш-функция распределяет ключи равномерно, а число ключей n не превышает m в разы, поиск, вставка и удаление выполняются в среднем за O(1).

Разные ключи неизбежно попадают в одну корзину — это **коллизия**. Способов разрешения коллизий два семейства:

- **Цепочки.** В каждой корзине — список ключей, попавших в неё. Поиск — проход по списку своей корзины. Коэффициент заполнения $\alpha = n/m$ может быть больше единицы; средняя длина цепочки равна α.
- **Открытая адресация.** Все ключи лежат прямо в массиве. При занятой ячейке пробуется следующая по заданной последовательности проб, пока не найдётся свободная или искомый ключ. Коэффициент заполнения строго меньше единицы, и при $\alpha \to 1$ число проб резко растёт.

Схема	Последовательность проб, i = 0, 1, 2, …	Особенности
Цепочки	одна корзина h mod m, список внутри	Терпит $\alpha > 1$, лишняя память на узлы списка
Линейное пробирование	(h + i) mod m	Хорошо работает с кешем процессора, но образует длинные кластеры
Квадратичное пробирование	(h + i(i+1)/2) mod m, m — степень двойки	Частный случай $h + c_1 i + c_2 i^2$ при $c_1 = c_2 = ½$; при $m = 2^p$ обходит все ячейки
Двойное хеширование	$(h + i \cdot h_2) \bmod m$, h_2 нечётно, m — степень двойки	Шаг зависит от ключа; нужна вторая хеш-функция

Рехеширование. Когда коэффициент заполнения превышает порог (на практике 0,5–0,7 для открытой адресации — при $\alpha = 0{,}9$ неудачный поиск линейным пробированием стоит около 50 проб — и 0,75–1 для цепочек), создаётся массив вдвое большего размера и все ключи переносятся в него заново — номер корзины зависит от m, поэтому копировать массив как есть нельзя. Отдельное рехеширование стоит O(n), но, поскольку размер удваивается, суммарная стоимость n вставок остаётся O(n), то есть амортизированно O(1) на вставку — тот же довод, что и для динамического массива.

Хеш-функции для строк

Во всех формулах s_i — очередной байт слова в кодировке UTF-8 как беззнаковые числа, вычисления ведутся в 64-битной беззнаковой арифметике (по модулю 2^{64}, переполнение — штатное поведение).

Функция	Вычисление	Комментарий
Полиномиальная	h = 0; для каждого байта: $h = h \cdot 31 + s_i$	Схема Горнера для многочлена по основанию 31; так считает хеш строки Java
djb2	h = 5381; для каждого байта: $h = h \cdot 33 + s_i$	Классическая функция Д. Бернстайна
FNV-1a	h = 14695981039346656037; для каждого байта: $h = h \text{ xor } s_i$; $h = h \cdot 1099511628211$	Сначала xor, затем умножение на простое число FNV

Для двойного хеширования вторая функция h_2 получается из той же хеш-функции варианта, например как (h >> 32) | 1: старшие биты дают шаг, а принудительно выставленный младший бит делает шаг нечётным и взаимно простым с $m = 2^p$. Хеш-функции для таблиц не являются криптографическими: они быстрые и хорошо перемешивают типичные данные, но не защищены от целенаправленного подбора коллизий. На этом построена атака HashDoS — подача ключей, сводящих таблицу к линейному поиску.

Три способа выбрать k ближайших

Для каждого запроса есть n кандидатов — пар (расстояние, номер строки). Нужно получить k наименьших в порядке возрастания.

- **Полная сортировка.** Отсортировать все n кандидатов и взять первые k. Просто и надёжно, но тратит O(n log n) на упорядочивание тех $n - k$ элементов, которые не нужны.
- **Куча размера k.** Поддерживать max-кучу из k лучших кандидатов, просмотренных до сих пор. Очередной кандидат сравнивается с вершиной — худшим из лучших; если он лучше, вершина заменяется и просеивается вниз. Время O(n log k), дополнительная память O(k). В конце k элементов кучи упорядочиваются извлечением максимума.
- **Быстрый выбор** (алгоритм Хоара). Разбиение, как в быстрой сортировке, но рекурсия идёт только в ту часть, где лежит граница k. В среднем O(n): размеры обрабатываемых частей убывают геометрически. В худшем случае — $O(n^2)$, если опорный элемент раз за разом оказывается крайним. После выбора k наименьших их нужно ещё упорядочить, это O(k log k).

Способ	Время на запрос	Худший случай	Доп. память
Полная сортировка	O(n log n)	O(n log n)	O(n) или O(log n)
Куча размера k	O(n log k)	O(n log k)	O(k)
Быстрый выбор	O(n + k log k)	$O(n^2)$	O(1) или O(log n)

Худший случай быстрого выбора достигается легко: при опорном элементе «первый» или «последний» достаточно уже упорядоченного входа. В этой работе он возникает естественно — если все письма выборки одинаковы, расстояния совпадают, и кандидаты приходят упорядоченными по номеру строки. Защита — случайный опорный элемент или медиана трёх.

Полный порядок кандидатов

Если сравнивать кандидатов только по расстоянию, при равных расстояниях три способа вернут разные наборы соседей: сортировка слиянием сохранит исходный порядок, куча и быстрый выбор — нет. Поэтому кандидаты сравниваются по паре **(расстояние, номер строки)**. Этот порядок полный — двух равных кандидатов не бывает, — и ответ однозначен независимо от способа.

Обратите внимание, что это не вопрос устойчивости сортировки: устойчивая сортировка лишь маскирует ошибку в сравнении, а куча и быстрый выбор её проявляют.

Условие задачи

Дана обучающая выборка писем с метками и набор писем-запросов. Для каждого запроса требуется найти k ближайших обучающих писем по расстоянию из раздела «Расстояние между письмами» и отнести запрос к классу большинства соседей. Словарь слов строится в вашей хеш-таблице из ДЗ1 с хеш-функцией по варианту Р3; выбор k ближайших выполняется одним из трёх способов, который задаётся при запуске.

Формат входных данных

Один текстовый файл: строка с числом k, маркер обучающей части, строки вида «метка || текст», маркер части запросов и по строке на запрос. Пустые строки игнорируются, номера строк считаются по файлу с единицы. Пример:

```
K 3
TRAIN
spam || Купите со скидкой сегодня!
ham || Встреча сегодня в 10:00
spam || Скидка только сегодня, купите
ham || Отчёт по встрече завтра
QUERY
скидка сегодня
отчёт: встреча завтра
```

Выход — строка на запрос: номер запроса, итоговая метка и соседи в виде «номер строки:расстояние» по возрастанию:

```
Q 1 spam 5:2 3:4 4:5
Q 2 ham 6:3 4:6 3:7
```

Точные правила — какие байты считаются буквами слова и как приводятся к нижнему регистру, как разрешается ничья голосов, какие строки считаются ошибочными и как о них сообщать — заданы в файле `io-format.md` в каталоге работы. Он является частью задания: программа, расходящаяся с ним в мелочах, не проходит автоматическую проверку.

Обратите внимание, что «скидкой» и «скидка» — разные слова: мешок слов ничего не знает о морфологии.

Базовые требования к программе

- Хеш-таблица — ваша из ДЗ1 (та же схема разрешения коллизий), подключённая как модуль; хеш-функция — по варианту Р3 и передаётся таблице параметром. Библиотечные словари и хеш-таблицы (`std::unordered_map`, `dict`, `HashMap` и аналоги) для словаря слов запрещены; использовать их разрешается только как эталон для самопроверки, и в отчёте это должно быть оговорено.
- Таблица растёт: при превышении порога коэффициента заполнения выполняется рехеширование. Порог выбирается самостоятельно и обосновывается в отчёте.
- Куча и быстрый выбор реализованы самостоятельно. Библиотечные очереди с приоритетом и функции выбора (`std::priority_queue`, `heapq`, `std::nth_element`, `std::partial_sort`) запрещены. Для способа полной сортировки библиотечная сортировка разрешена — это эталон, с которым сравниваются два других способа.
- Вывод трёх способов совпадает на любом входе.

- Быстрый выбор не деградирует до квадратичного времени на упорядоченном входе.
- Программа не завершается аварийно и не зависает ни на каком входном файле, включая пустой, двоичный, с некорректной кодировкой и с огромными строками.
- Предусмотрен режим замера, выводящий время выбора k ближайших отдельно от чтения файла, построения словаря и подсчёта расстояний. Результаты замера не попадают в выходной файл.
- Куча оформлена как отдельный переиспользуемый модуль: она вернётся в работе 4 (очередь с приоритетом в алгоритме Дейкстры). Хеш-таблица уже оформлена модулем в ДЗ1 и вернётся в работе 6 (частоты биграмм).

Формат запуска фиксирован, поскольку проверка автоматическая:

```
solution --method sort   вход.txt выход.txt
solution --method heap   вход.txt выход.txt
solution --method select вход.txt выход.txt
```

Проверка результата

Перед сдачей прогоните автоматическую проверку из каталога репозитория:

```
python3 check.py r3-spam-filter
```

Каждый публичный тест запускается трижды — по разу на каждый способ, — и вывод каждого запуска сравнивается с эталоном. Тесты устойчивости проверяют, что программа не падает и укладывается в лимит времени; среди них есть входы, на которых быстрый выбор с наивным опорным элементом работает квадратично. Ключ `--bench` даёт прикидочную картину роста времени всего процесса; замеров по протоколу курса он не заменяет.

Вместе с вариантом выдаётся персональный входной файл `personal.in`. Вывод программы на нём сдаётся вместе с отчётом и на приёмке сверяется с эталоном; при приёмке добавляются также скрытые тесты на краевые случаи. Зелёный результат по публичным тестам необходим, но не достаточен.

Варианты

Схема разрешения коллизий — та же, что в вашем варианте ДЗ1: в этой работе используется ваша хеш-таблица из ДЗ1. Вариант Р3 задаёт хеш-функцию и seed генератора данных; их сообщает преподаватель вместе с персональным входом.

№	Хеш-функция	На что обратить внимание
1	Полиномиальная	Распределение по корзинам при m — степени двойки и при простом m (лекция 6, раздел 2.2); сравнение с функцией из ДЗ1, отличающейся только модулем
2	djb2	Чем умножение на 33 хуже перемешивания для коротких слов; влияет ли начальное значение 5381 на коллизии слов одной длины
3	FNV-1a	Почему xor перед умножением перемешивает лучше; цена перемешивания на коротких словах

Для замеров используйте генератор `gen.py` со своим seed: `python3 gen.py <n> <seed> --queries 20 --k 10`. Генератор раздувает выборку до сотен тысяч синтетических писем трёх классов, словарь при этом растёт вместе с n.

Дополнительное задание (повышенный уровень)

Обязательно для выхода на отличную оценку. Выбрать не менее двух пунктов.

- **Другие расстояния.** Реализовать расстояние Жаккара $1 - |A \cap B|/|A \cup B|$ и косинусное сходство на частотах слов. Сравнить с базовым расстоянием на письмах из публичных тестов, предъявить письмо, которое базовое расстояние классифицирует неверно, а нормированное — верно, и объяснить почему.
- **TF-IDF и стоп-слова.** Взвесить слова по формуле TF-IDF, вычисленной вручную, удалить частые служебные слова («и», «в», «на», «это»…). Показать, как изменились соседи и метки.
- **Три класса и качество классификации.** Разметка ham / spam / phish, матрица ошибок, отдельно ложноположительные (нормальное письмо попало в спам) и ложноотрицательные (спам или фишинг прошёл фильтр) срабатывания при разных k. Объяснить, какая ошибка дороже для почтового фильтра и почему.
- **Атака уклонения.** Написать спам или фишинговое письмо, которое собственный фильтр относит к ham, сохранив смысл письма для человека. Назвать, какое свойство метрики или мешка слов было использовано, и предложить защиту.

Требования к отчёту

Отчёт оформляется по общему шаблону курса и содержит четыре обязательных раздела.

- **Асимптотика** трёх способов выбора и хеш-таблицы по времени и памяти с обоснованием, а не со ссылкой на таблицы из методички. Отдельно — амортизированная стоимость вставки с рехешированием.
- **Замеры** по протоколу курса: не менее шести размеров выборки, растущих удвоением, не менее семи повторов, медиана, указание разброса. Два графика времени одного выбора: при фиксированном малом k и при k, растущем вместе с n (например, $k = n/10$); на каждом — три способа. Проверка гипотез роста нормировкой $t(n)/f(n)$. Отдельно — доли времени построения словаря, подсчёта расстояний и выбора: имеет ли смысл ускорять выбор, если доминирует что-то другое.
- **Худший случай** для быстрого выбора (неудачный опорный элемент): механизм, конструкция входа, замер. Для хеш-таблицы — связь с атакой из ДЗ1: что произойдёт со спам-фильтром, если противник пришлёт письма из слов, подобранных под хеш-функцию вашего варианта, и какая часть работы программы при этом деградирует.
- **Масштабирование:** что откажет первым при росте выборки в тысячу раз — память под словарь и письма, подсчёт расстояний или выбор.

Отдельным пунктом — статистика собственной хеш-таблицы на словаре персонального входа: размер словаря, итоговое m, коэффициент заполнения, средняя и максимальная длина цепочки (или среднее и максимальное число проб), число рехешированний.

Контрольные вопросы

- Почему для выбора k наименьших используется max-куча, а не min-куча?
- Почему быстрый выбор работает за $O(n)$ в среднем, но за $O(n^2)$ в худшем случае? Как выбор опорного элемента влияет на худший случай?
- При каких соотношениях k и n куча размера k перестаёт выигрывать у полной сортировки?

- Почему порядок соседей задан парой (расстояние, номер строки)? Что сломается, если сравнивать только расстояния, и почему ошибка не видна при сортировке слиянием?
- Что происходит с хеш-таблицей с открытой адресацией при коэффициенте заполнения, близком к единице? Почему цепочки переносят это мягче?
- Почему рехеширование не ухудшает асимптотику вставки? Что изменится, если увеличивать таблицу не вдвое, а на фиксированное число ячеек?
- Почему при m — степени двойки качество младших битов хеш-функции особенно важно?
- Чем хеш-функция для хеш-таблицы отличается от криптографической? Защищает ли криптографическая хеш-функция от HashDoS сама по себе?
- Почему при базовом расстоянии короткие письма оказываются «ближе ко всем»? Как это исправляют нормированные метрики?
- Как выбор k влияет на классификацию? Что происходит при k, равном размеру выборки?